

AI 543: Foundation of Online Machine Learning

Homework 3: Bandit Algorithms

Mrinal Bharadwaj

March 2, 2026

1 Overview

This report presents the implementation and evaluation of classical bandit algorithms in simulated environments. The experiments cover:

- **Task 1 (Multi-Armed Bandits):** Explore Then Commit, UCB, Thompson Sampling
- **Task 2 (Linear Bandits):** LinUCB, LinTS

All results can be reproduced by running:

```
python RunAll.py
```

2 Task 1: Multi-Armed Bandits (60 points)

2.1 Environment Setup

To simulate a K -armed bandit, we use the following settings:

- `actionset = "basis_vector"`: article feature vectors are basis vectors $e_0, e_1, \dots, e_{K-1}$, making arms independent.
- `K = n_articles = context_dimension = 10`
- `NoiseScale =  $\sigma = 0.1$` (standard deviation of Gaussian noise)
- `n_users = 10`
- `testing_iterations = 10,000`
- `poolArticleSize = None` (all arms available each round)

Since feature vectors are basis vectors, the reward for arm i selected by user with parameter θ is:

$$r_t = \theta^\top e_i + \epsilon_t = \theta_i + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2)$$

Each arm has an independent mean reward $\mu_i = \theta_i$, which the algorithms must learn.

2.2 Explore Then Commit (20 points)

2.2.1 Algorithm Description

Input: Number of arms K , exploration parameter m , arm set $\mathcal{A}$.

Algorithm 1 Explore Then Commit

- 1: **Exploration Phase:** For each arm $i \in \{1, \dots, K\}$, pull arm i exactly m times.
- 2: Record the empirical mean reward for each arm:

$$\hat{\mu}_i = \frac{1}{m} \sum_{s=1}^m r_{i,s}$$

- 3: **Commit Phase:** For all remaining rounds $t > mK$, always select:

$$a_t = \arg \max_{i \in \{1, \dots, K\}} \hat{\mu}_i$$

The total exploration phase lasts $m \times K$ rounds. After that, the algorithm commits to the arm with the highest observed mean forever.

2.2.2 Results

We test $m \in \{1, 2, 3, 5, 8, 10, 15, 20, 50, 100, 200, 500\}$ (12 values).

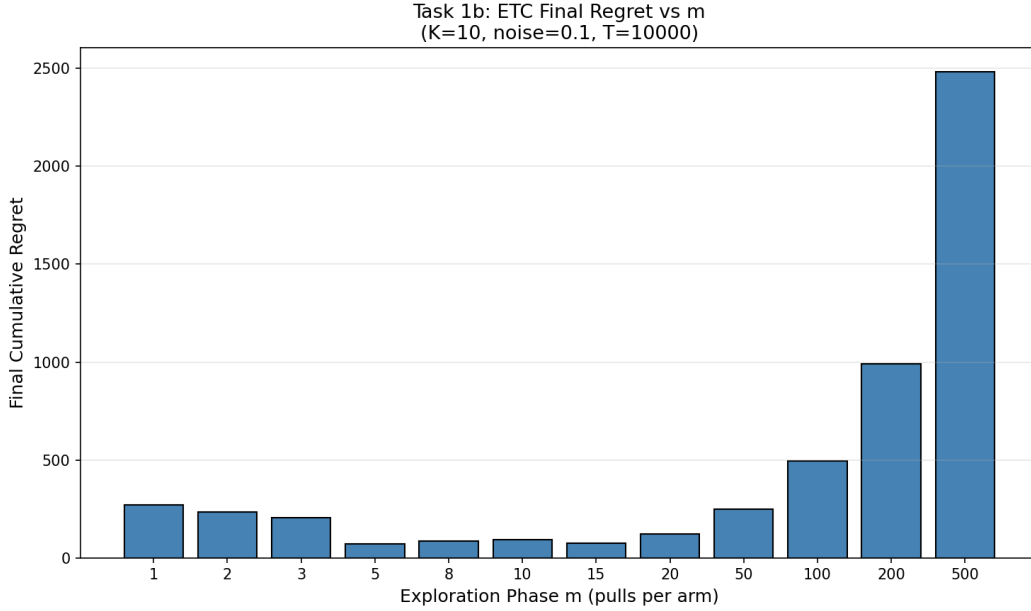


Figure 1: ETC: Final cumulative regret as a function of m .

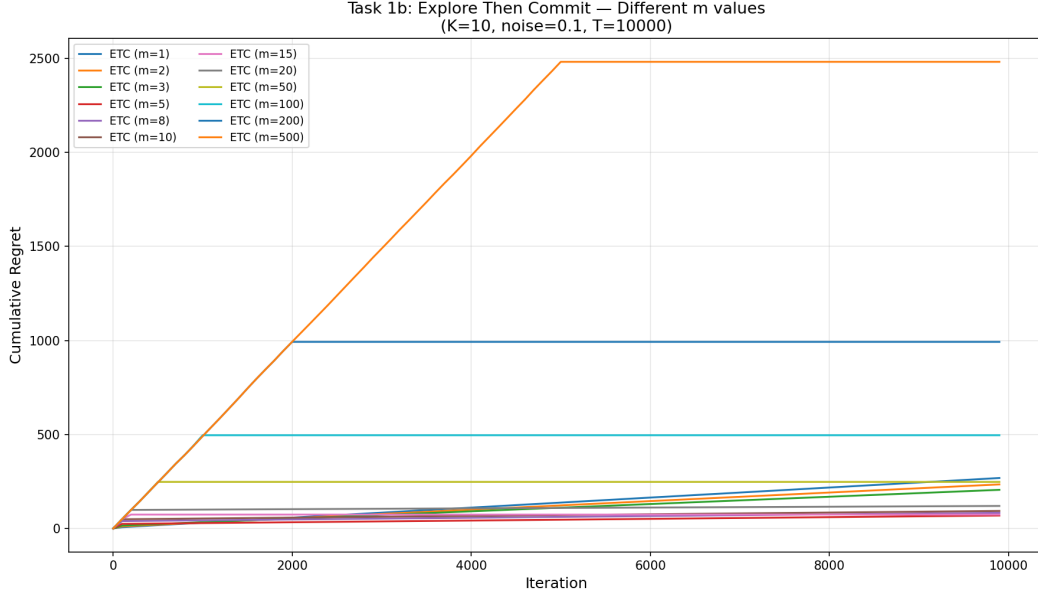


Figure 2: ETC: Cumulative regret over time for all m values.

Table 1: ETC final cumulative regret for different m values.

m (pulls per arm)	Final Cumulative Regret
1	268.91
2	234.86
3	206.01
5	69.30
8	84.05
10	93.89
15	74.45
20	120.30
50	248.17
100	496.34
200	992.68
500	2481.69

2.2.3 Analysis: Which m is best and why?

The best m is $\mathbf{m = 5}$ with a final cumulative regret of 69.30. The results show a clear U-shaped tradeoff:

- **m too small** (e.g., $m = 1, 2$): Each arm is explored very few times. With noise ($\sigma = 0.1$), the empirical mean $\hat{\mu}_i$ is a poor estimate of the true mean μ_i . The algorithm frequently commits to a *suboptimal* arm, leading to high regret that accumulates linearly forever after the commit phase.
- **m too large** (e.g., $m = 100, 200, 500$): The exploration phase itself takes $m \times K$ rounds. With $m = 500$ and $K = 10$, the algorithm spends 5,000 out of 10,000 rounds just exploring,

pulling suboptimal arms many times unnecessarily. This wastes rounds that could be spent exploiting the best arm.

- **m just right** ($m \approx 5$): The algorithm gets a sufficiently accurate estimate of each arm's mean (with $m = 5$ observations and $\sigma = 0.1$, the standard error is $\sigma/\sqrt{m} = 0.045$, which is small enough to distinguish arms) while spending only 50 rounds exploring.

2.3 UCB (20 points)

2.3.1 Algorithm Description

Input: Number of arms K , exploration parameter α .

The UCB (Upper Confidence Bound) algorithm selects the arm that maximizes an optimistic estimate of the mean reward. The UCB index for arm i at time t is:

$$\text{UCB}_i(t) = \hat{\mu}_i(t) + \alpha \sqrt{\frac{\ln t}{N_i(t)}} \quad (1)$$

where:

- $\hat{\mu}_i(t)$ is the empirical mean reward of arm i up to time t
- $N_i(t)$ is the number of times arm i has been pulled up to time t
- t is the total number of pulls so far
- α is the exploration parameter controlling the width of the confidence bound

Algorithm 2 UCB

- 1: Pull each arm once (initialization).
 - 2: **for** $t = K + 1, K + 2, \dots, T$ **do**
 - 3: For each arm i , compute $\text{UCB}_i(t)$ using Eq. (1).
 - 4: Select $a_t = \arg \max_i \text{UCB}_i(t)$.
 - 5: Observe reward r_t and update $\hat{\mu}_{a_t}$ and N_{a_t} .
 - 6: **end for**
-

The key idea is “optimism in the face of uncertainty”: arms with fewer observations receive a larger exploration bonus, encouraging the algorithm to try under-explored arms.

2.3.2 Results

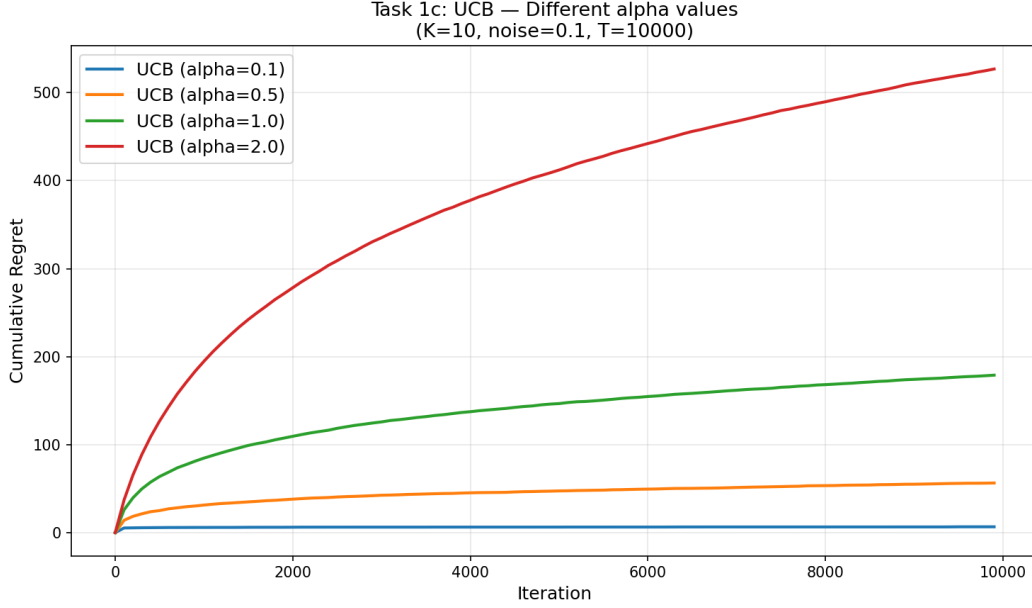


Figure 3: UCB: Cumulative regret with different α values.

Table 2: UCB final cumulative regret for different α values.

α	Final Cumulative Regret
0.1	7.02
0.5	56.74
1.0	179.13
2.0	526.61

Smaller α reduces unnecessary exploration. With low noise ($\sigma = 0.1$), the algorithm quickly identifies the best arm and $\alpha = 0.1$ is sufficient. The regret of UCB grows logarithmically $O(\ln T)$, which is visible in the flattening curves.

2.4 Thompson Sampling (20 points)

2.4.1 Algorithm Description

Input: Number of arms K , known noise variance σ^2 .

Thompson Sampling uses a Bayesian approach. We maintain a posterior distribution over each arm's mean reward μ_i and sample from it to make decisions.

Prior:

$$\mu_i \sim \mathcal{N}(0, 1) \quad \text{for each arm } i$$

Likelihood:

$$r_t \mid \mu_i \sim \mathcal{N}(\mu_i, \sigma^2)$$

Posterior after observing n_i rewards with sum $S_i = \sum_{s=1}^{n_i} r_{i,s}$:

$$\tau_i^2 = \frac{1}{\frac{1}{\sigma_0^2} + \frac{n_i}{\sigma^2}} \quad (2)$$

$$m_i = \tau_i^2 \left(\frac{\mu_0}{\sigma_0^2} + \frac{S_i}{\sigma^2} \right) \quad (3)$$

$$\mu_i \mid \text{data} \sim \mathcal{N}(m_i, \tau_i^2) \quad (4)$$

where $\mu_0 = 0$ and $\sigma_0^2 = 1$ are the prior mean and variance.

Algorithm 3 Thompson Sampling (Gaussian)

```

1: for  $t = 1, 2, \dots, T$  do
2:   for each arm  $i$  in the available set do
3:     Compute posterior mean  $m_i$  and variance  $\tau_i^2$  using Eqs. (2)–(3).
4:     Sample  $\tilde{\mu}_i \sim \mathcal{N}(m_i, \tau_i^2)$ .
5:   end for
6:   Select  $a_t = \arg \max_i \tilde{\mu}_i$ .
7:   Observe reward  $r_t$  and update  $S_{a_t} \leftarrow S_{a_t} + r_t$ ,  $n_{a_t} \leftarrow n_{a_t} + 1$ .
8: end for
```

2.4.2 Results

Thompson Sampling achieves the lowest regret among all MAB algorithms.

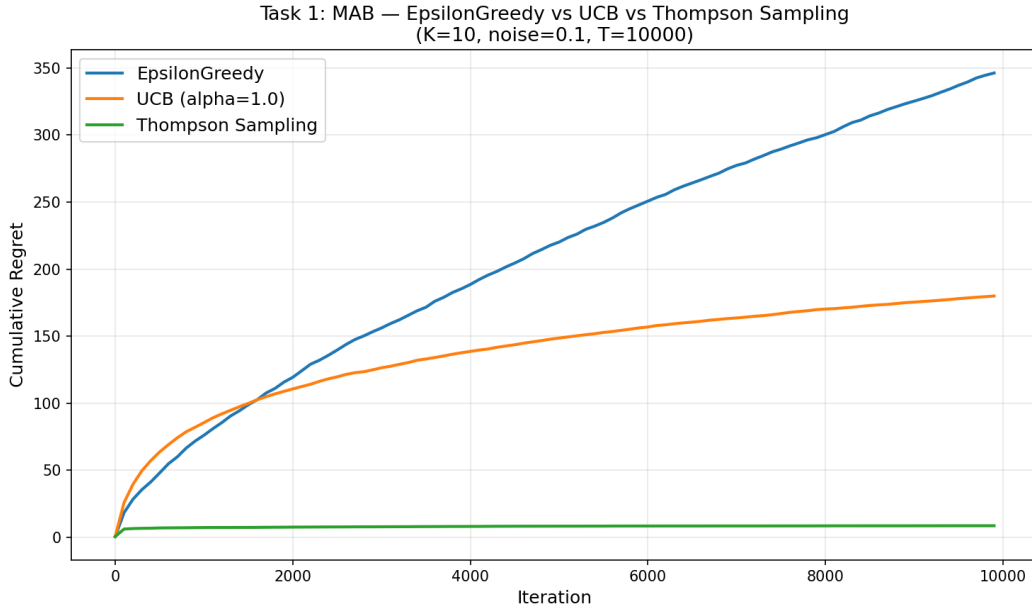


Figure 4: MAB comparison: EpsilonGreedy vs UCB vs Thompson Sampling.

Table 3: Final cumulative regret comparison for MAB algorithms.

Algorithm	Final Cumulative Regret
EpsilonGreedy	346.20
UCB ($\alpha = 1.0$)	179.92
Thompson Sampling	8.64

Thompson Sampling dramatically outperforms both EpsilonGreedy and UCB. Its Bayesian approach naturally balances exploration and exploitation: arms with high uncertainty have wide posteriors that occasionally produce high samples (triggering exploration), while well-understood good arms consistently produce high samples (enabling exploitation). The posterior quickly concentrates around the true mean, so exploration diminishes rapidly.

3 Task 2: Linear Bandits (40 points)

3.1 Environment Setup

For linear bandits, we use:

- `actionset = "random"`: article feature vectors are random unit vectors
- `context_dimension = d = 25`
- `n_articles = 25`
- `poolArticleSize = 10` (10 random articles available per round)
- `NoiseScale =  $\sigma = 0.1$`
- `n_users = 10`
- `testing_iterations = 10,000`

The reward model is:

$$r_t = \theta^\top x_{a_t} + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma^2)$$

where $\theta \in \mathbb{R}^d$ is the user's unknown preference vector and $x_{a_t} \in \mathbb{R}^d$ is the feature vector of the selected arm. Unlike MAB, observing reward from one arm provides information about *all* arms through the shared θ .

3.2 LinUCB (20 points)

3.2.1 Algorithm Description

Input: Feature dimension d , regularization parameter λ , exploration parameter α .

Parameter Estimation (Ridge Regression): We maintain:

$$A_t = \lambda I + \sum_{s=1}^t x_{a_s} x_{a_s}^\top \in \mathbb{R}^{d \times d} \tag{5}$$

$$b_t = \sum_{s=1}^t x_{a_s} r_s \in \mathbb{R}^d \tag{6}$$

$$\hat{\theta}_t = A_t^{-1} b_t \tag{7}$$

UCB for arm selection:

$$\text{UCB}(x) = \hat{\theta}_t^\top x + \alpha \sqrt{x^\top A_t^{-1} x} \quad (8)$$

Algorithm 4 LinUCB

```

1: Initialize  $A \leftarrow \lambda I_d$ ,  $b \leftarrow \mathbf{0}_d$ 
2: for  $t = 1, 2, \dots, T$  do
3:   Compute  $\hat{\theta}_t = A^{-1}b$ 
4:   for each arm  $x$  in the available set do
5:     Compute  $\text{UCB}(x) = \hat{\theta}_t^\top x + \alpha \sqrt{x^\top A^{-1} x}$ 
6:   end for
7:   Select  $a_t = \arg \max_x \text{UCB}(x)$ 
8:   Observe reward  $r_t$ 
9:   Update:  $A \leftarrow A + x_{a_t} x_{a_t}^\top$ ,  $b \leftarrow b + x_{a_t} r_t$ 
10: end for

```

The term $\sqrt{x^\top A_t^{-1} x}$ measures the uncertainty of the predicted reward in the direction x . Directions that have been observed frequently contribute more to A_t , reducing uncertainty in those directions.

Hyperparameters used: $\lambda = 1.0$, $\alpha \in \{0.1, 0.5, 1.0\}$.

3.2.2 Results

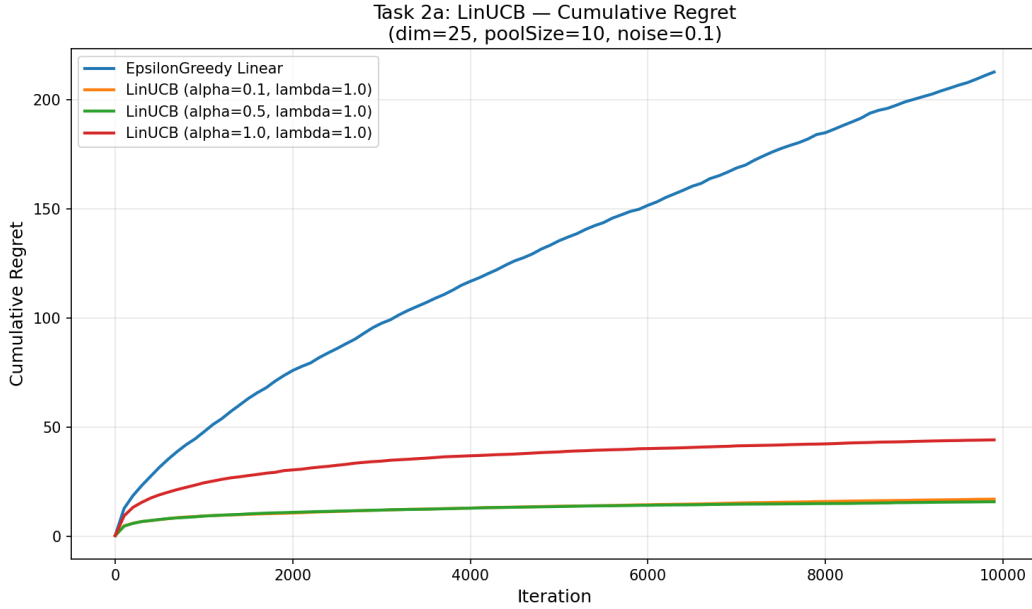


Figure 5: LinUCB: Cumulative regret with different α values.

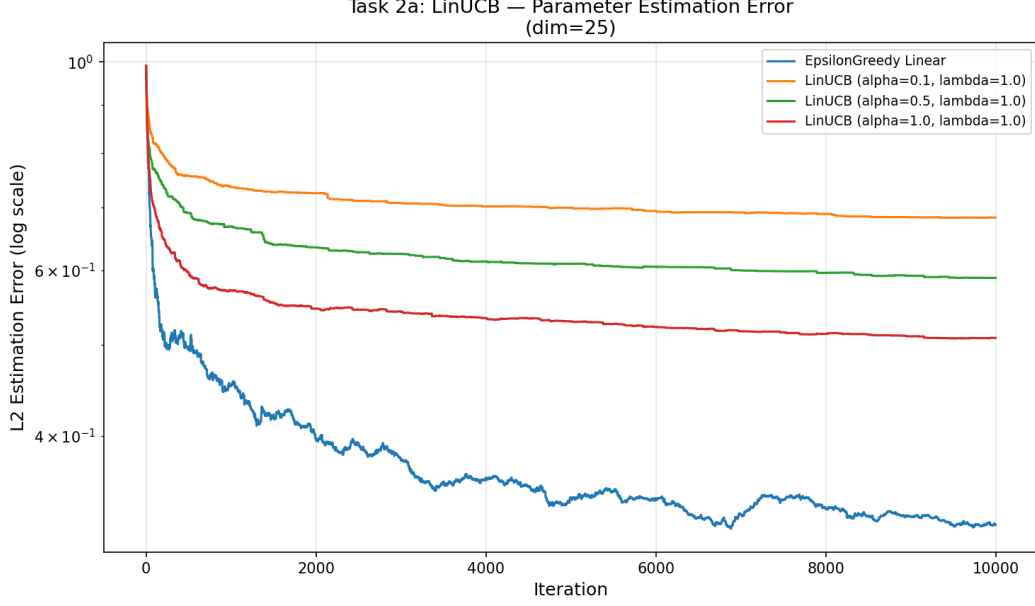


Figure 6: LinUCB: Parameter estimation error $\|\theta - \hat{\theta}\|_2$ over time (log scale).

Table 4: LinUCB final cumulative regret.

α	λ	Final Cumulative Regret
0.1	1.0	16.93
0.5	1.0	15.73
1.0	1.0	44.11
EpsilonGreedy Linear		212.80

LinUCB with $\alpha = 0.5$ achieves the best performance. The parameter estimation error decreases steadily over time as A_t accumulates more observations, and LinUCB dramatically outperforms EpsilonGreedy by leveraging the linear structure of the reward function.

3.3 LinTS (20 points)

3.3.1 Algorithm Description

Input: Feature dimension d , regularization parameter λ , sampling scale v .

Linear Thompson Sampling uses the same ridge regression as LinUCB but replaces the confidence bound with posterior sampling.

Prior:

$$\theta \sim \mathcal{N}\left(\mathbf{0}, \frac{1}{\lambda} I_d\right)$$

Posterior after t observations:

$$\theta \mid \text{data} \sim \mathcal{N}(\mu_t, V_t) \quad (9)$$

$$V_t = A_t^{-1} = \left(\lambda I + \sum_{s=1}^t x_{a_s} x_{a_s}^\top \right)^{-1} \quad (10)$$

$$\mu_t = V_t b_t = A_t^{-1} \sum_{s=1}^t x_{a_s} r_s \quad (11)$$

Decision:

$$\tilde{\theta}_t \sim \mathcal{N}(\mu_t, v^2 V_t) \quad (12)$$

$$a_t = \arg \max_{x \in \mathcal{A}_t} \tilde{\theta}_t^\top x \quad (13)$$

Algorithm 5 LinTS

- 1: Initialize $A \leftarrow \lambda I_d$, $b \leftarrow \mathbf{0}_d$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: Compute $\mu_t = A^{-1}b$ and $V_t = A^{-1}$
 - 4: Sample $\tilde{\theta}_t \sim \mathcal{N}(\mu_t, v^2 V_t)$
 - 5: Select $a_t = \arg \max_x \tilde{\theta}_t^\top x$
 - 6: Observe reward r_t
 - 7: Update: $A \leftarrow A + x_{a_t} x_{a_t}^\top$, $b \leftarrow b + x_{a_t} r_t$
 - 8: **end for**
-

The parameter v controls the amount of exploration: theoretically $v^2 = \sigma^2$, but in practice it can be tuned.

Hyperparameters used: $\lambda = 1.0$, $\sigma = 0.1$, $v \in \{0.01, 0.1, 0.5\}$.

3.3.2 Results

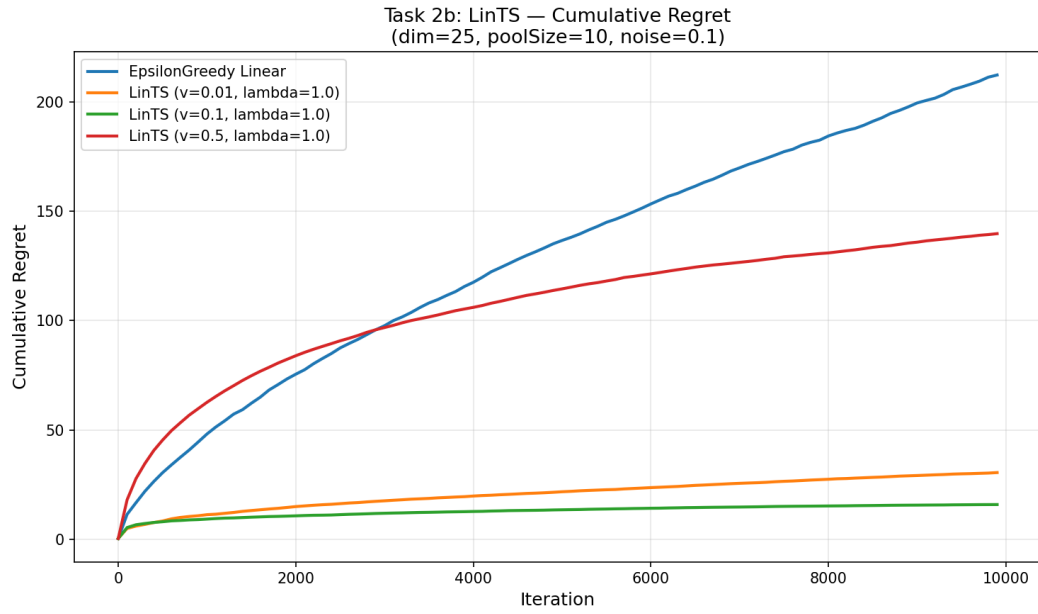


Figure 7: LinTS: Cumulative regret with different v values.

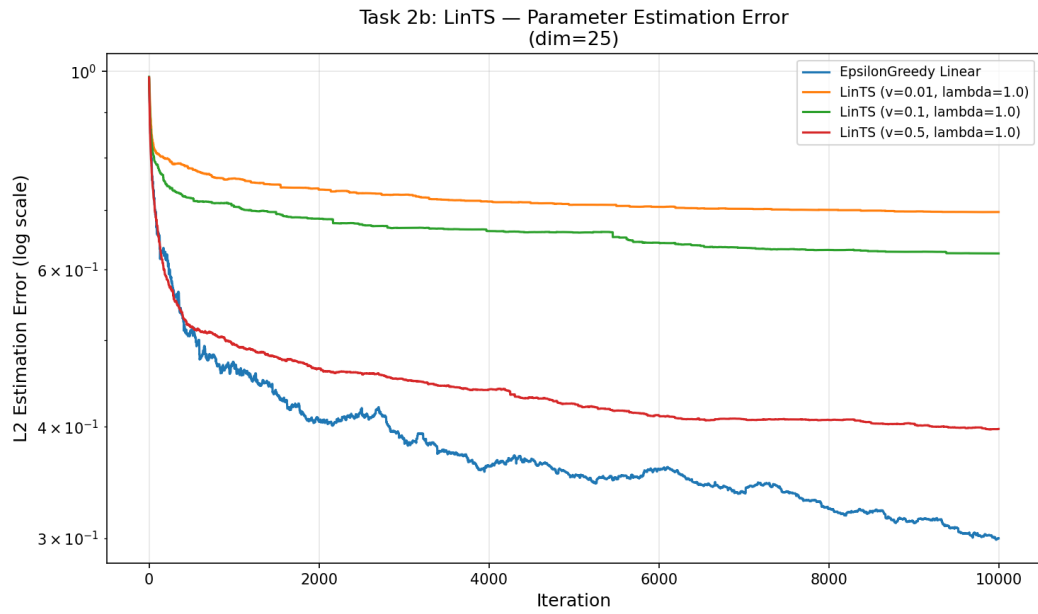


Figure 8: LinTS: Parameter estimation error over time (log scale).

Table 5: LinTS final cumulative regret.

v	λ	σ	Final Cumulative Regret
0.01	1.0	0.1	30.49
0.1	1.0	0.1	15.86
0.5	1.0	0.1	139.70
EpsilonGreedy Linear			212.23

$v = 0.1 = \sigma$ (matching the noise standard deviation) gives the best performance, consistent with theory. When v is too small ($v = 0.01$), the algorithm under-explores and converges slowly. When v is too large ($v = 0.5$), it over-explores.

3.4 Combined Comparison: LinUCB vs LinTS vs EpsilonGreedy

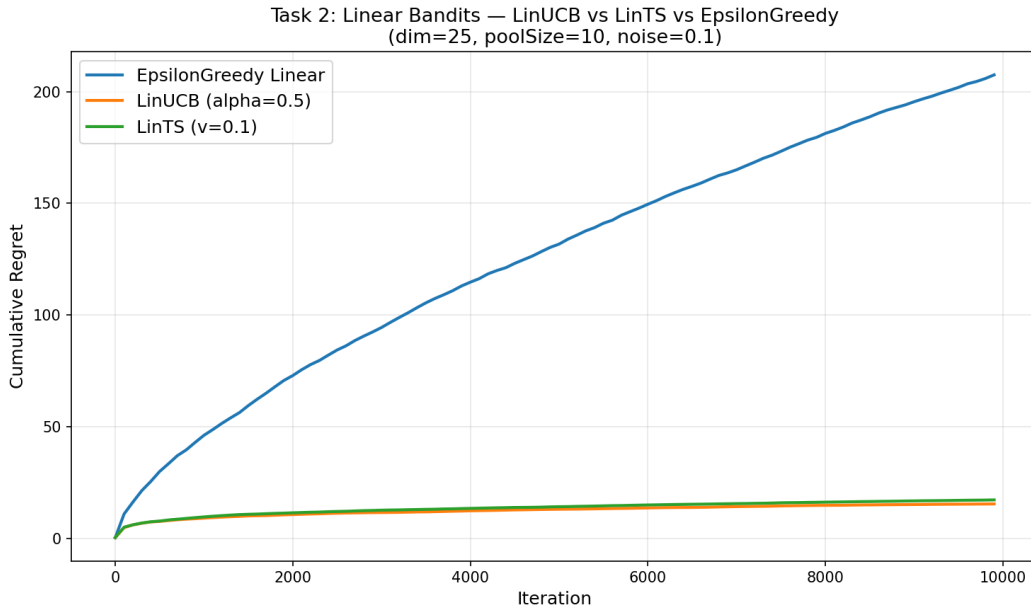


Figure 9: Linear bandits: LinUCB vs LinTS vs EpsilonGreedy cumulative regret.

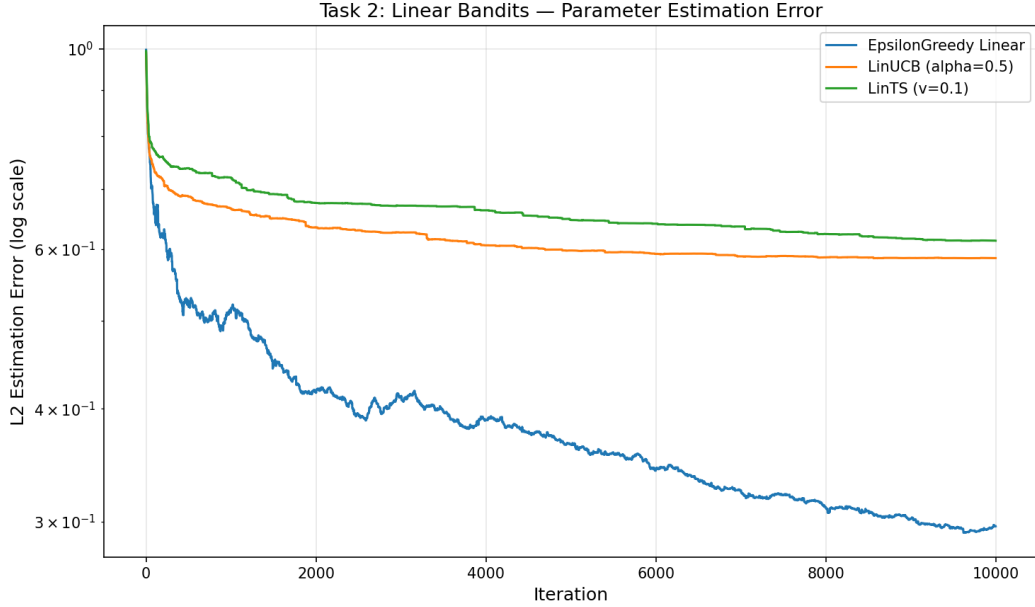


Figure 10: Linear bandits: Parameter estimation error comparison.

Table 6: Final comparison of linear bandit algorithms.

Algorithm	Final Cumulative Regret
EpsilonGreedy Linear	207.52
LinUCB ($\alpha = 0.5$)	15.40
LinTS ($v = 0.1$)	17.21

Both LinUCB and LinTS significantly outperform EpsilonGreedy (by more than 10 $\times$). LinUCB and LinTS achieve comparable performance. The parameter estimation error (Figure 10) decreases over time for all algorithms, confirming that the ridge regression estimator converges to the true θ .

4 Conclusion

Key takeaways from the experiments:

1. **Explore Then Commit** has a clear exploration-exploitation tradeoff controlled by m . The optimal m depends on the noise level and time horizon. It is simple but inflexible compared to adaptive methods.
2. **UCB** adaptively balances exploration and exploitation through confidence bounds. It achieves $O(\sqrt{K \ln T})$ regret and requires no distributional assumptions.
3. **Thompson Sampling** is the best-performing MAB algorithm, achieving near-optimal regret. Its Bayesian nature provides a natural and effective exploration mechanism.
4. **LinUCB** and **LinTS** leverage the linear reward structure to share information across arms, achieving dramatically lower regret than MAB baselines in the linear setting. Both achieve

comparable performance, with LinUCB being slightly deterministic and LinTS being stochastic.

References

- [1] T. Lattimore and C. Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.